# RDFIA 1 - cd

Julien Kordes

October 2024

# Table des matières

**1 Introduction** **2**

**2 Partie 1 - Introduction to convolutional neural network** **2**

**3 Partie 2 - Training from scratch of the model** **4**
3.1 Network architecture . . . . . 4
3.2 Network learning . . . . . 6

**4 Partie 3 - Results improvements** **9**
4.1 Standardization of examples . . . . . 9
4.2 Increase in the number of training examples by data increase . . . . . 11
4.3 Variants on the optimization algorithm . . . . . 12
4.4 Regularization of the network by dropout . . . . . 13
4.5 Use of batch normalization . . . . . 16

# 1 Introduction

This practical focuses on Convolutional neural network (CNN). It helps us build a simple CNN and understanding its functioning. The first part enhances the different aspects of the CNN layers such as conv2D and Pooling (stride, padding, etc...). The second part deals with the computation of the model and the third part studies the effect of augmentation on the model's performance.

# 2 Partie 1 - Introduction to convolutional neural network

**Question 1**

Height : $\frac{x+2p-k}{s} + 1$

Width : $\frac{y+2p-k}{s} + 1$

Output of size : $Height * Width * z$

There are $k * k * z$ weights to learn (+1 if we consider the bias)

For a fully connected layer between an input of size $x * y * z$ and an output of size $\left(\frac{x+2p-k}{s} + 1\right) * \left(\frac{y+2p-k}{s} + 1\right) * z$, we would need to reshape the tensors as 1 dimensional and the number of weight would be (without considering bias):

$$(x * y * z) * \left( (\frac{x + 2p - k}{s} + 1) * (\frac{y + 2p - k}{s} + 1) * z \right)$$

**Question 2**

Using fully-connected layers with images requires a large number of parameters, as each pixel has its own weight. Additionally, these layers are sensitive to variations in images, such as simple translations.

On the other hand, convolutional layers help to address these issues. By applying a shared set of weights (or filters) across the entire image, they enable the learning and detection of local patterns in different areas. They also combine features hierarchically: lower layers capture low-level details (like edges and textures), while higher layers represent more complex features, identifying objects. However, convolutional layers are limited in capturing global context, as they mainly focus on local patterns and can struggle to link distant parts of an image.

**Question 3**

Spatial pooling increases translation invariance by summarizing local regions, reducing the emphasis on exact feature locations and acting as a regularization method.

It also lowers the spatial dimensions of feature maps, making the computational costs decrease.

**Question 4**

In a convolutional network, a convolution layer can adapt to any input size, performing convolutions across the full span of an image without requiring a specific input size as a hyperparameter. As convolution and pooling layers stack up, the output size is entirely determined by the input size. However, when the output moves to fully connected layers, it must first be "flattened" into a vector, and the fully connected layers need to know the input size beforehand for setting up weights. This requirement can create a challenge when the input image is larger than expected, as the fully connected layers require a fixed input size.

**Question 5**

We can think of a fully-connected layer as a special case of a 1D convolution with a Kernel size equal to the entire input size, a stride of 1 and no padding. It results in a single output since the kernel covers the entire image in one time. In this case, the convolution's kernel has a shape $(1, n)$ or $(m, n)$ (if there are multiple output channels), which is equivalent to the weight matrix $W$ in a dense layer. So, for input $x$ and weight matrix $W$, the output of this "convolutional" operation is: $y = Wx$ This operation is identical to a fully-connected layer with no bias.

**Question 6**

Replacing fully connected layers with convolutional equivalents allows a neural network to process images of varying sizes rather than just the original input size until the computation of the loss. However, these layers will not act as fully-connected layers since it requires to know the exact input size (which is equal to the kernel size) as we saw on the previous question. When the inputs are larger images, convolutional layers produce larger feature maps that go through the network, with the final convolutional layers applying filters across the full spatial dimensions of these maps. This results in a spatial score map on which we have to apply Average Pooling to compute the loss.

**Question 7**

The general formula to calculate the receptive field is :

$$R_l = R_{l-1} + (k_l - 1) \times \prod_{i=1}^{l-1} s_i$$

where $k_l$ is the kernel size, $s_l$ is the stride and $R_l$ is the receptive field of the layer $l$.

If we apply this formula to the convoluational layers we get:
For the first Convolutional Layer:

$$R_1 = 1 + (5 - 1) = 5$$

We need to know the receptive field of the first pooling layer before calculating the one of the the second convolutional layer.
For the first pooling layer :

$$R_2 = 5 + (2 - 1) * 1 = 6$$

Then for the second convolutional layer (with a stride of 1) :

$$R_3 = 6 + (5 - 1) * 1 * 2 = 14$$

Concerning the deeper Layers: as we go deeper into the network, the receptive fields grow significantly, capturing a larger portion of the original image. This expansion allows neurons in deeper layers to integrate spatially larger and more complex patterns from earlier layers, effectively "seeing" more of the image at once.
To conclude, early layers detect basic patterns like edges or textures, while deeper layers represent more abstract features such as shapes and objects, this can eventually allow an interpretation of the image in the final layers.

# 3 Partie 2 - Training from scratch of the model

## 3.1 Network architecture

**Question 8**
If we use the equations of the question 1, and we impose that $H_{in} = H_{out}$, we get :

$$\frac{H_{in} - k + 2p}{s} + 1 = H_{out}(= H_{in})$$

The usual choice is to take $s = 1$ which leads us to $p = \frac{k-1}{2}$ and with $k = 5$ we get $p = 2$
So we need **a stride of 1 and a padding of 2 for the convolutional layers**.

**Question 9**
We use the same method but with $H_{out} = \frac{H_{in}}{2}$.

Here the usual choice is to take $s = 2$ and this gives us $p = \frac{k}{2} - 1$ and with $k = 2$ we get $p = 0$.

So we need **a stride of 2 and a padding of 0 for the maxpooling layers**.

**Question 10**

Calculation of the weights layer by layer considering 1 bias per layer with an input size of $32 * 32 * 3$:

- For the first layer conv1:

  We get $32 * (5 * 5 * 3 + 1) = 2432$ weights and an output size of $32 * 32 * 32$

- For the second layer pool1:

  There is no learnable parameters and the output size is $16 * 16 * 32$

- For the third layer conv2:

  We get $64 * (5 * 5 * 32 + 1) = 51264$ weights and an output size of $16 * 16 * 64$

- For the fourth layer pool2:

  There is no learnable parameters and the output size is $8 * 8 * 64$

- For the fifth layer conv3:

  We get $64 * (5 * 5 * 64 + 1) = 102464$ weights and an output size of $8 * 8 * 64$

- For the sixth layer pool3:

  There is no learnable parameters and the output size is $4 * 4 * 64$

- For the seventh layer fc4:

  We get $(4 * 4 * 64 + 1) * 1000 = 1,025,000$ weights and an output size of $1 * 1000$

- For the eighth and last layer fc5:

  We get $(1 * 1000 + 1) * 10 = 10,010$ weigths and an output size of $1 * 10$

**Question 11**

The makes a total of 1,191,170 weights to learn Given a dataset with 50,000 training samples and 10,000 test samples, we get a total of 60,000 examples. Therefor we have to be cautious about the risk of overfitting.

**Question 12**

In contrast, the Bag of Words (BoW) and Support Vector Machine (SVM) approach, when using a vocabulary of 1,000 visual words and 10 classes, has 10,010 parameters (as calculated with $(1000+1)\times 10(1000+1)\times 10$).

The much smaller parameter space in BoW + SVM makes it less prone to overfitting and computationally lighter. However it relies on the quality of the handcrafted features extracted in the BoW representation, which may limit its performance on complex image data compared to CNNs.

## 3.2 Network learning

**Question 14**

The code differs in that we don't pass an optimizer to the epoch() function (for the evaluation part). As a result, the network is not trained on test examples, and no backward pass occurs to compute or update the loss.

**Question 16**

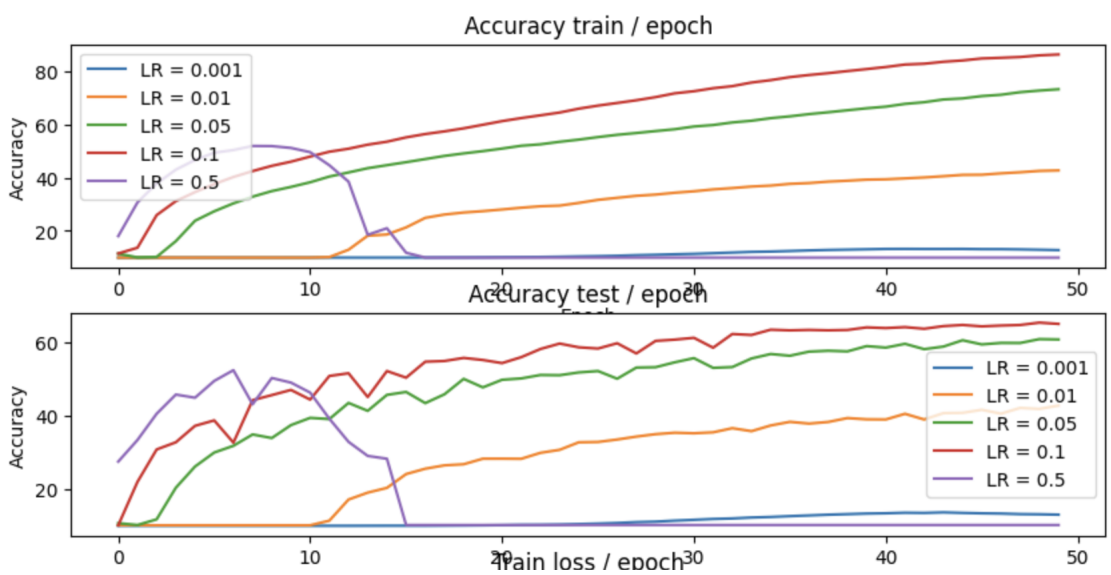


Figure 1: Influence of learning rate on the accuracy of our model on the CIFAR dataset

The learning rate criticaly affects the performance of the model. The conclusions are the same as in the last practical (1 - ab) where we saw that a higher learning rate can speed up convergence by making larger parameter updates. However if

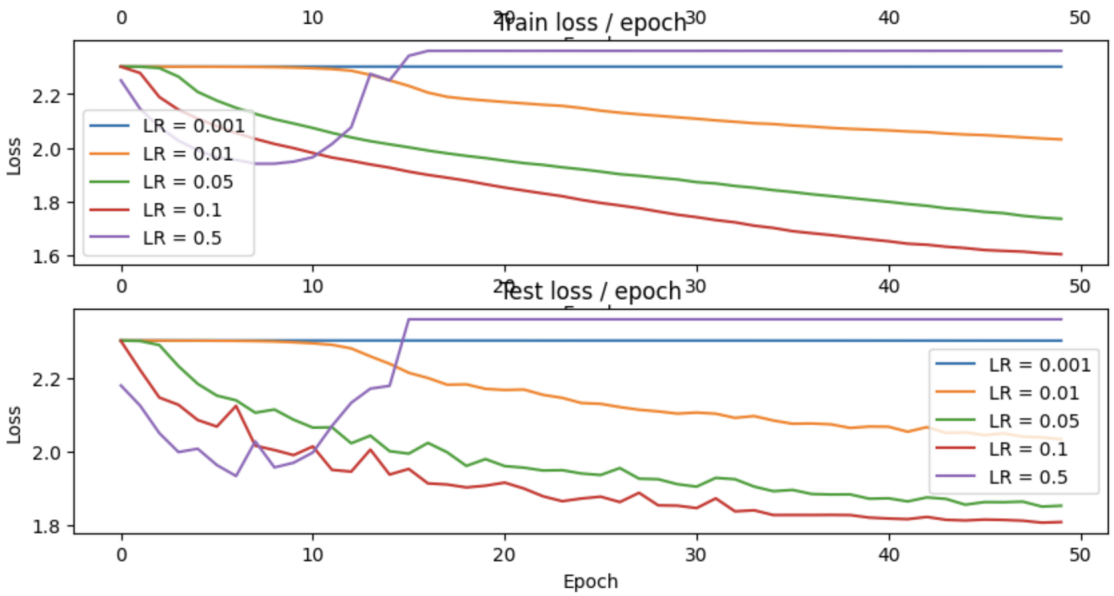


Figure 2: Influence of learning rate on the loss of our model on the CIFAR dataset

it is too high, it may cause issues like overshooting or divergence. On the other hand, a smaller learning rate allows more careful exploration of the loss landscape, potentially avoiding shallow minima but can lead to slower convergence or getting stuck.

That is exactly what we can see on the FIGURE 1 and the FIGURE 2 where we have to find a compromise between careful exploration, convergence and the danger of getting stuck (lr = 0.001).

To test the batch size I only used 20 epochs since it was enough to determinate their influence on the model's performance. Using a larger batch size provides a more accurate gradient estimate, which can result in more stable and consistent training steps but requires more memory. Indeed as we can see on the FIGURE 4, the test loss becomes lower for bigger batch sizes after a few epochs. Smaller batch sizes, on the other hand, introduce more noise in the gradient and it appears to lead to better results for early epochs as the training and test accuracy are higher for Batch sizes of 32 and 64.

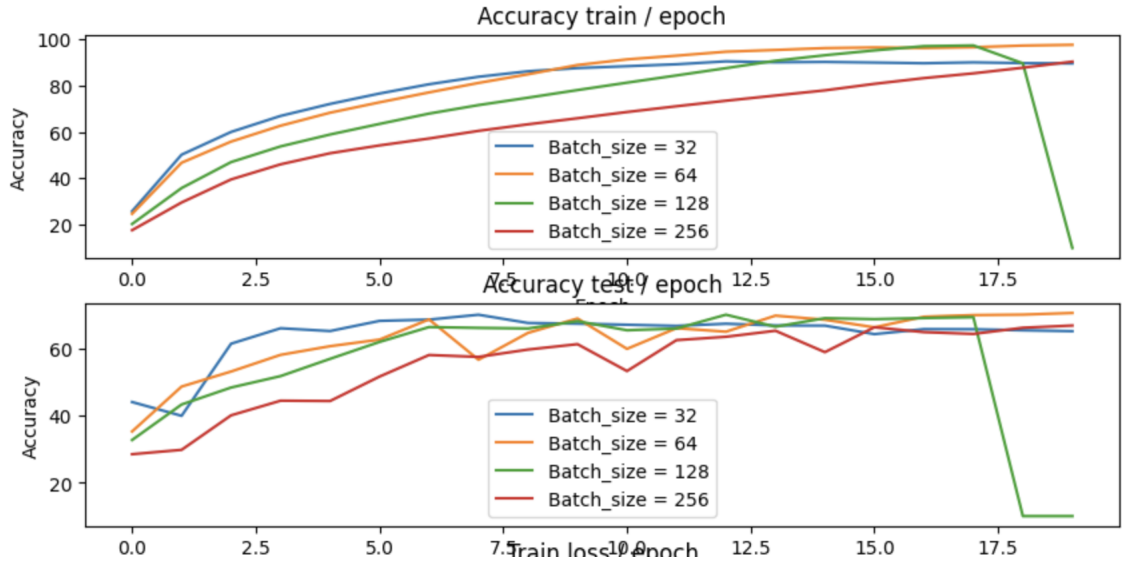


Figure 3: Influence of batch size on the accuracy of our model on the CIFAR dataset

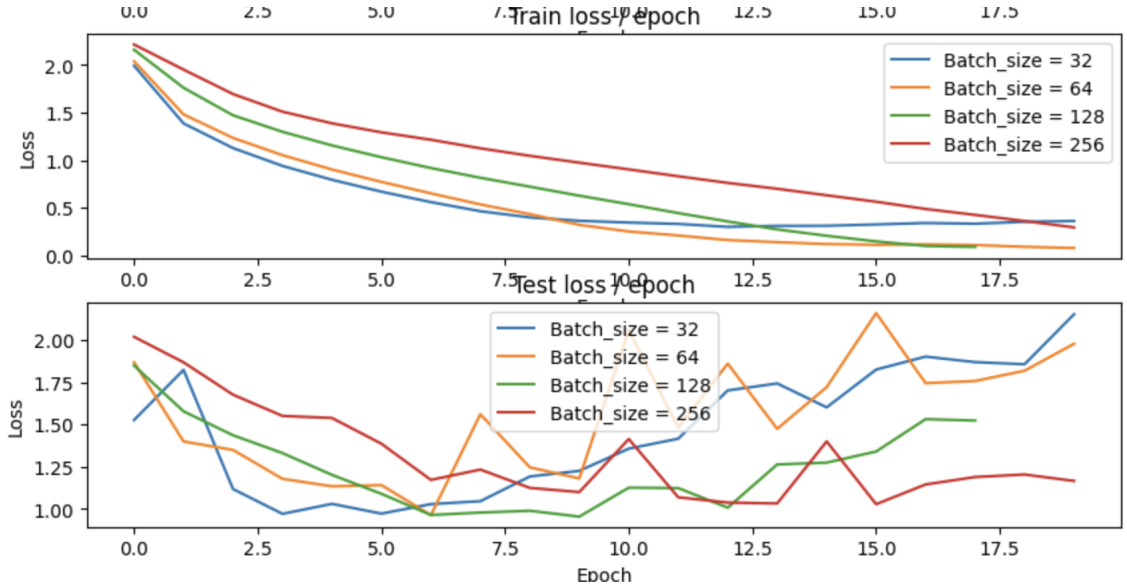


Figure 4: Influence of batch size on the loss of our model on the CIFAR dataset

**Question 17**

As we can see on the FIGURE 5, at the start of the first epoch, the test accuracy is higher than the training accuracy. This can be explained by the fact that the

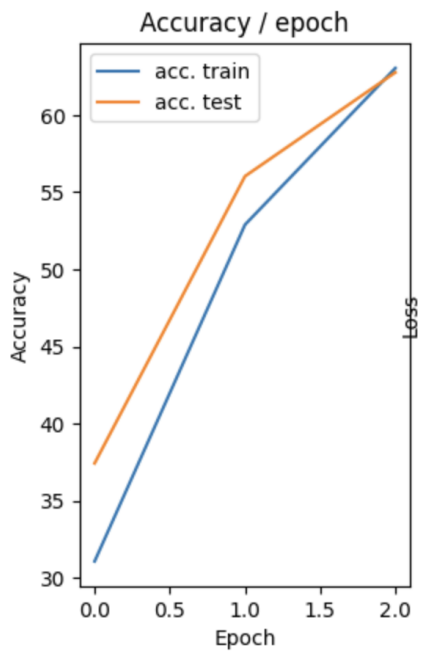


Figure 5: First epoch of the training

training accuracy and the training loss are calculated at each batch so it creates more fluctuation than for the test set. This can cause the test accuracy to be higher at the beginning of the learning procedure because it is only calculated at the end of the first epoch.

**Question 18**

The phenomen that is occurring is overfitting. Around the 30th epoch, test accuracy levels off at about 60%, while training accuracy keeps improving. This shows that the model is performing better on the training data but struggles to generalize to new, unseen test images.

# 4 Partie 3 - Results improvements

## 4.1 Standardization of examples

**Question 19**

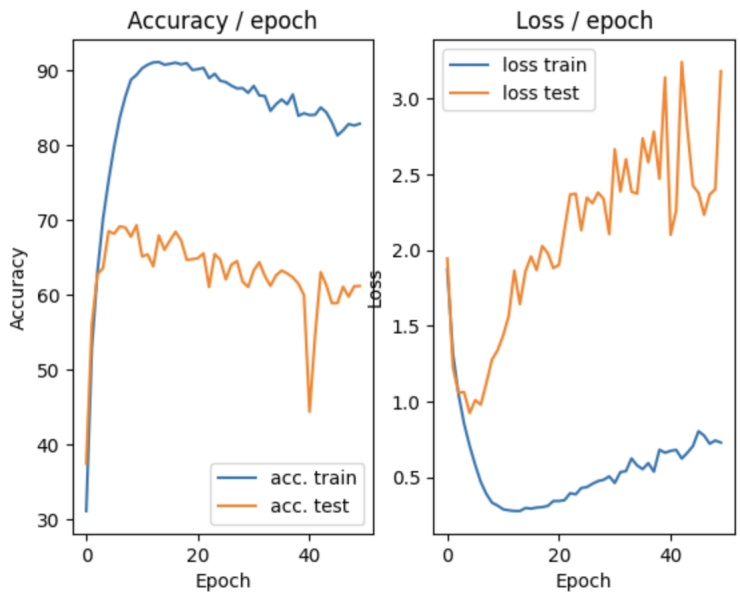


Figure 6: Accuracy and loss of the model using standardization of examples

As we can see on the FIGURE 6, the standardization of examples helped the model to converge faster. Indeed, the standardization prevents any single feature from dominating due to its scale, leading to faster and often smoother convergence. As a matter of fact, the curve is really smooth for the first 20 epochs. After those 20 epochs, the accuracy starts to drop which is a sign of overfitting as we discussed on the question 18. The standardization of examples doesn't appear to prevent the model from this issue.

**Question 20**

This ensures that the model assesses its performance on data processed in the same way as the training data, without having prior knowledge of the specific characteristics of the validation set. If these statistics were computed using the validation set, it would allow the model to gain insights into the validation data. Such an approach is crucial for an unbiased evaluation of the model's performance and its ability to generalize to new, unseen data.

## 4.2 Increase in the number of training examples by data increase

**Question 22**

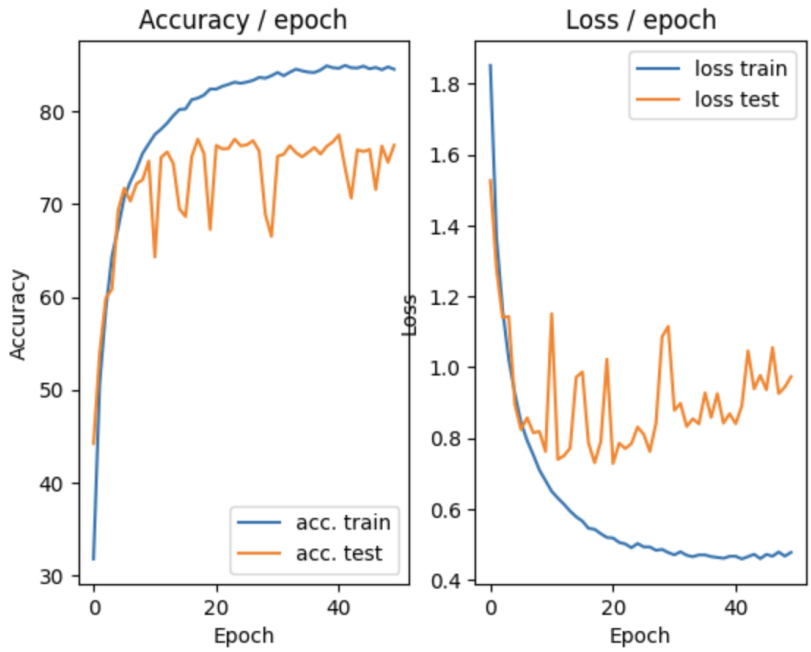


Figure 7: Accuracy and loss of the model with an increased number of data examples

Artificially increasing our dataset should create a model that generalizes better and reduce sensitivity to the specific details of training images by introducing more variety. Additionally, random cropping and flipping add a layer of perturbation to the data, helping the model build robustness to variations it may encounter in real-world situations. Indeed the FIGURE 7 shows that the overfitting has been reduced since the last method: the test and training accuracy don't drop anymore which is a good point. However, we still observe some instability in the test loss and accuracy as the data increasing method didn't manage to solve the problem entirely.

**Question 23**

For many tasks, horizontal flipping does not change the subject's identity. Indeed, a cat remains recognizable when flipped. This technique is effective in these cases, adding robustness to the model without confusing the model. However, in tasks

where orientation is essential such as writing recognition, where flipping a character can change its meaning, the horizontal symmetry approach is not usable.

**Question 24**

Transformations may introduce unnatural features, leading the model to learn incorrect patterns. Transformation of the dataset can also sometimes worsen class imbalances if transformations are applied uniformly, increasing the representation of already dominant classes. Moreover if we use a complex transformation method, it can become computationally heavy without necessary providing new and relevant information to the model.

## 4.3 Variants on the optimization algorithm

**Question 26**

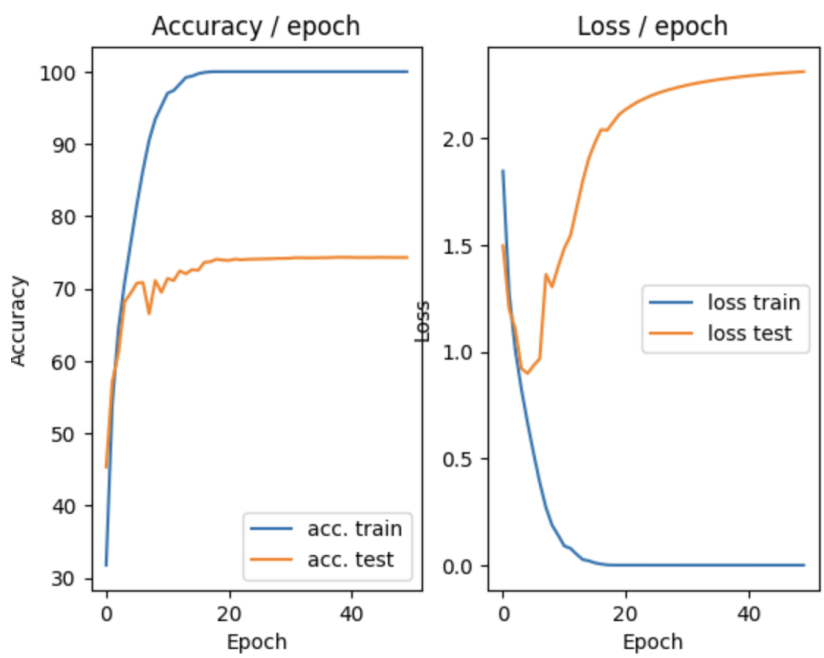


Figure 8: Accuracy and loss of the model with variants on the optimization algorithm

Using a learning rate scheduler allows the model to adapt more effectively to the problem by controlling how quickly or slowly it learns. This approach can help the

model avoid areas where accuracy plateaus due to slow learning, while promoting stable and efficient training. As we can see on the FIGURE 8, the training accuracy reached is higher and close to 100% and the training curves are way smoother than with other methods. However, the overfitting issue still seems to occur since the model is not able to expand his learning to the test dataset better than with an accuracy of 70% after the first 10 epochs.

**Question 27**

Exponential decrease scheduling for the learning rate enhances training by adapting dynamically to different stages. Early in training, a higher learning rate allows faster progress as the model explores the parameter space, while later, a reduced rate enables finer adjustments to refine the model. This approach also adapts to variations in the loss function landscape, with larger steps for steep areas and smaller steps for flatter regions, improving efficiency. However we have to set the initial rate and decay factor carefully, as too fast a decay may limit learning, while too slow a decay can cause overshooting or instability near the minimum.

## 4.4 Regularization of the network by dropout

**Question 29**

The aim when using a dropout layer is to reduce overfitting. However, in the FIGURE 9 we can see that overfitting is still present as the accuracy starts to drop after a few epochs. Thuis can be explained by the fact that I used a pretty low value of p for this dropout layer. Indeed I chose a value of 0.2 but using a higher value should help reducing the overfitting by deactivating more neurons of the neural network. The influence of the parameter p is studied in the question 32.

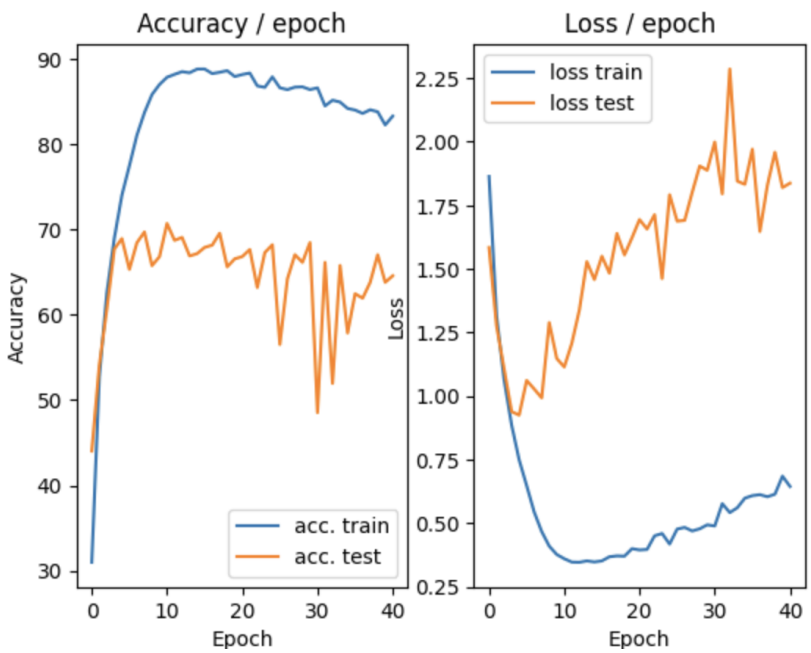


Figure 9: Accuracy and loss of the model with regularization of the network by dropout

**Question 30**

Regularization includes techniques that limit a model's learning capacity to reduce overfitting, where the model learns the training data too precisely and struggles to generalize to new data. These methods add a penalty to the loss function or adjust the training data or model structure to favor simpler, more generalizable models.

**Question 31**

Dropout is a regularization method that randomly "drops" certain neurons during training, preventing their activation to reduce co-dependence among neurons. This forces neurons to learn independent and robust features, avoiding over-reliance on specific connections. It also acts as a form of model averaging, as each training iteration uses a slightly different network, improving the model's robustness and generalization. By adding noise to the network, dropout helps prevent overfitting, similar to data augmentation, by reducing the model's tendency to memorize the training data.

**Question 32**

The Figure 10 and Figure 11 represent the influence of the parameter p on

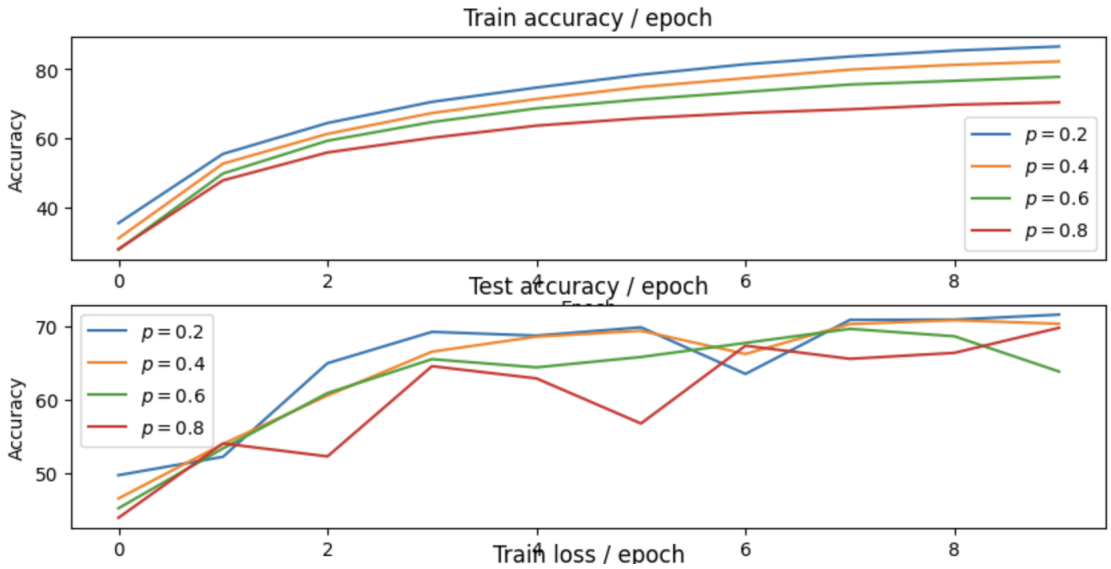


Figure 10: Influence of p on the accuracy of the model

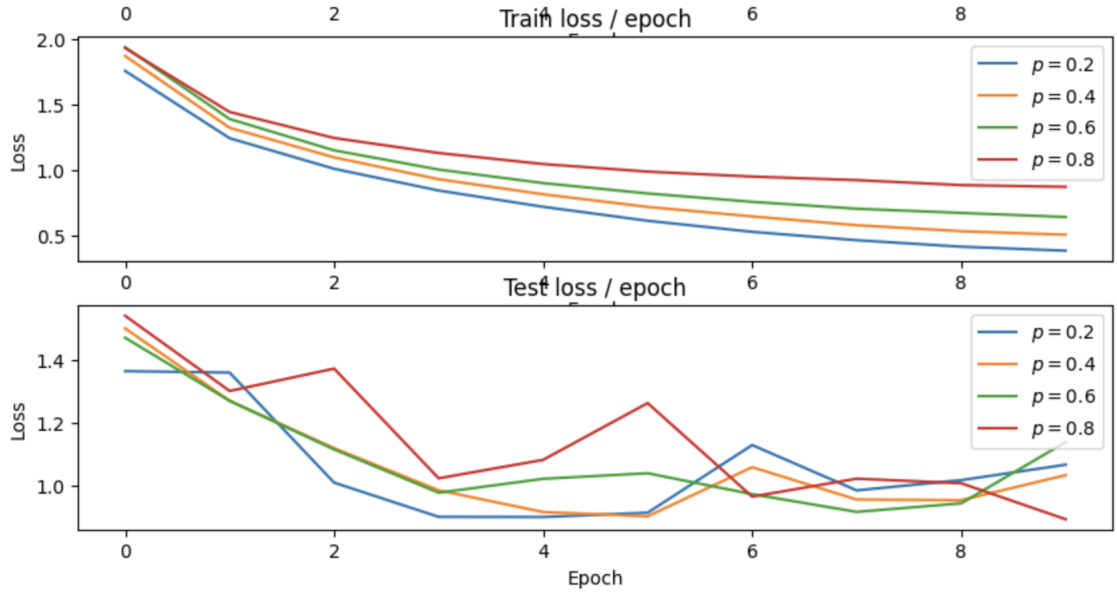


Figure 11: Influence of p on the loss of the model

the performance of our model. I only printed the 10 first epochs as I didn't have enough computation resources but here is what we can say. At the beginning of the learning, a lower value of p seems to be better since the neural network needs all

his neurons to learn efficiently from the data. However, as we can see on the test loss curve, the value of p = 0.8 seems to become better as the number of epochs increases: indeed this helps to reduce the overfitting when the training has already started.

**Question 33**

During training, dropout randomly deactivates a fraction of neurons in each batch based on a dropout rate. When testing, dropout is turned off so all neurons remain active. This approach helps the model generalize better while providing stable results on unseen data.

## 4.5 Use of batch normalization

**Question 34**

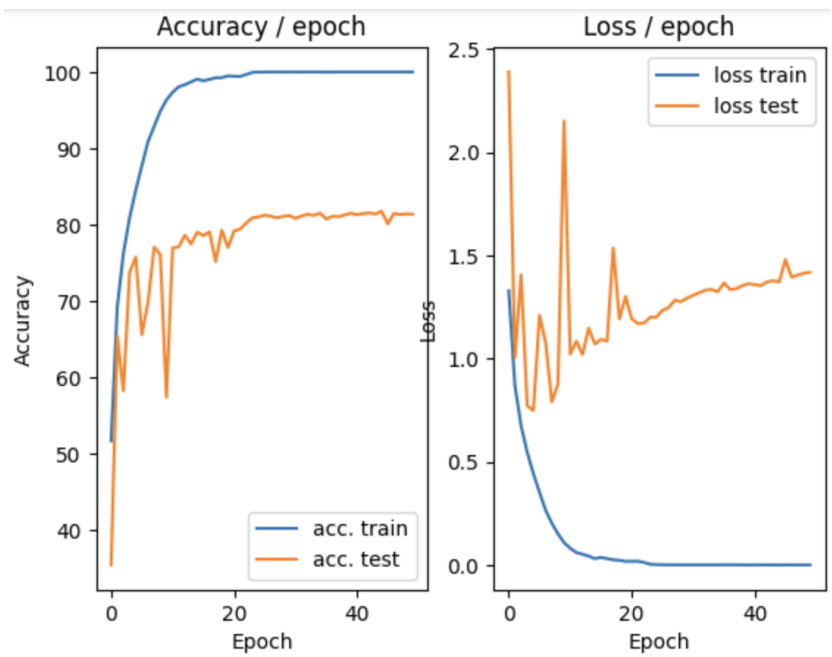


Figure 12: Accuracy and loss of the model using batch normalization

By stabilizing the distribution of inputs at each layer, it can speed up training and allow the model to converge in fewer epochs. Indeed it reaches a training accuracy

close to 100% pretty fast as we can see on the FIGURE 12. This results in a higher-quality model compared to training without batch normalization, as it helps the network to adapt faster and more effectively.